

Uniwersytet DSW Ideis
studia pierwszego stopnia

Kierunek studiów: Informatyka
Zakres kształcenia: Niestacjonarne

Barbara Składanowska
Katarzyna Tritchart
Tycjan Harczak

**Interaktywny Symulator Rdzenia i Pulpitu Operatorskiego Reaktora Jądrowego z Warstwą
Predykcji Anomalii**

Praca projektowa

Wrocław, 2026

1. Wstęp i tło projektowe

Gwałtowny rozwój komputerowych systemów wspomagania operatora oraz interfejsów HMI (Human-Machine Interface) umożliwia precyzyjne odwzorowywanie krytycznych procesów przemysłowych. W sektorze energetyki jądrowej symulatory czasu rzeczywistego stanowią fundamentalne narzędzie dydaktyczne i walidacyjne.

Niniejszy projekt przedstawia zintegrowany system symulacji fizyko-termicznej rdzenia reaktora jądrowego (strukturalnie inspirowanego reaktorami kanałowymi dużej mocy RBMK-1000). System łączy warstwę symulacji numerycznej (Python), przemysłową bazę danych szeregów czasowych (InfluxDB), asynchroniczny bufor wiadomości (Redis), zaawansowane środowisko wizualizacji i kontroli operatorskiej (Grafana SCADA Dashboard) oraz sztuczną inteligencję opartą na głębokich sieciach rekurencyjnych (PyTorch LSTM Autoencoder) realizującą zadanie predykcji nieliniowych anomalii stanów przejściowych rdzenia.

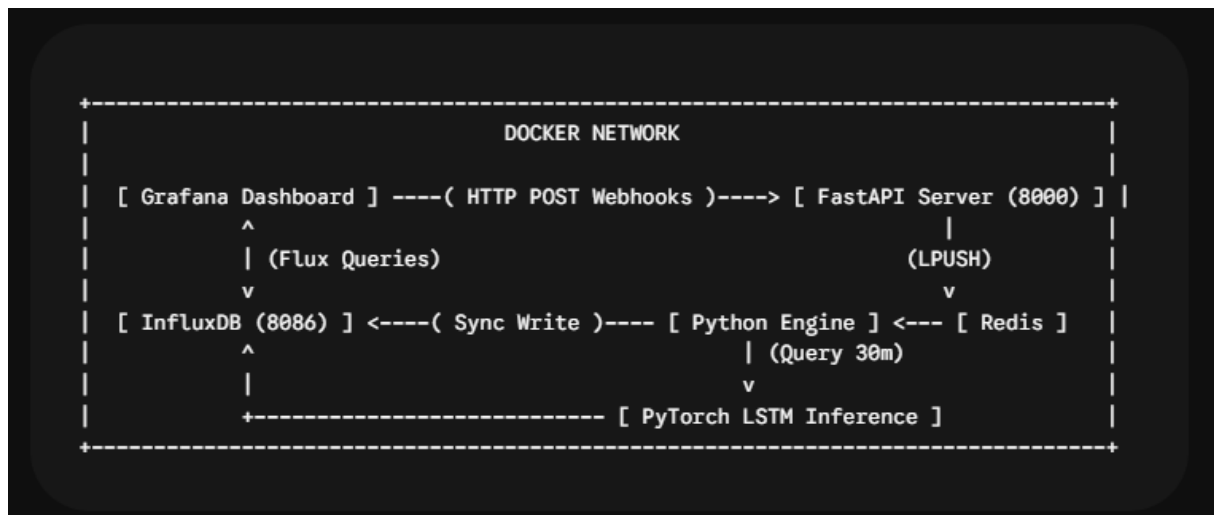
2. Cel projektu

Głównym celem projektu było zaprojektowanie, implementacja i testowanie wielowarstwowego, rozproszonego ekosystemu informatycznego, realizującego następujące zadania:

- Numeryczne modelowanie fizyki reaktora: Matematyczna symulacja sprzężeń zwrotnych (void coefficient, xenon poisoning) oraz bilansu cieplnego w rdzeniu z krokiem dyskretnym $\Delta t = 1.0$ s.
- Projekt ergonomicznego interfejsu HMI: Stworzenie w Grafanie pulpitu sterowniczego, umożliwiającego kontrolę prętów absorberów i chłodzenia w sposób intuicyjny, odwzorowujący fizyczne zachowanie mechanizmów.
- Asynchroniczna komunikacja i sterowanie zdarzeniami: Wdrożenie brokera Redis oraz FastAPI do bezopóźnieniowej obsługi komend operatorskich i ochrony przed zakleszczeniami procesów.
- Detekcja anomalii za pomocą Deep Learning: Implementacja potoku danych opartego na algorytmie Autoencodera LSTM (sieci neuronowej wykrywającej anomalie w stanach przejściowych reaktora na podstawie błędów rekonstrukcji sygnałów).

3. Architektura systemowa i przepływ danych

System opiera się na architekturze rozproszonej, sterowanej zdarzeniami (Event-Driven Architecture), podzielonej na odseparowane mikrousługi zaimplementowane w środowisku kontenerowym Docker.



3.1. Warstwa API i obsługa Webhooków (api.py)

Za przyjmowanie poleceń od operatora odpowiada aplikacja sieciowa zaimplementowana w frameworku FastAPI (api.py). Serwer wystawia cztery dedykowane punkty końcowe (endpoints) typu POST: /prety, /water, /in_water_temp oraz /az5. W celu umożliwienia bezpośredniej integracji z interfejsem Grafany uruchomionym na innych adresach sieciowych, w warstwie FastAPI zaimplementowano middleware CORS (Cross-Origin Resource Sharing) z pełną autoryzacją nagłówków i metod (allow_origins=["*"]). Dane wejściowe są automatycznie walidowane strukturalnie przez klasę GrafanaData opartą na bibliotece Pydantic.

3.2. Asynchroniczne buforowanie danych w bazie Redis (redis_client.py)

Aby uniknąć wąskich gardeł (bottlenecks) i zjawiska zakleszczenia wątków przy jednoczesnym zapisie i odczycie, FastAPI po odebraniu pakietu JSON dokonuje natychmiastowego zepchnięcia wartości do asynchronicznych kolejek in-memory bazy Redis (redis_client.py) za pomocą operacji LPUSH:

```
r.lpush("rods_movement", data.value)
r.lpush("water_flow", data.value)
```

Podczas uruchamiania systemu (@app.on_event("startup")), serwer automatycznie czyści pozostałości w pamięci podręcznej Redisa (r.delete), gwarantując, że symulacja zawsze startuje z czystym stanem początkowym.

3.3. Pętla główna i synchronizacja stanu (app.py, database.py)

Główny proces sterujący (app.py) działa w nieskończonej pętli ze stałym taktowaniem dyskretnym $\Delta t = 1.0$ s. W każdej iteracji metoda regulation_workflow wykonuje operację

RPOP na bazie Redis (implementacja kolejki FIFO). Jeśli w Redisie znajdują się nowe zdarzenia od operatora, system przełącza się w tryb sterowania ręcznego. W przypadku braku nowych zdarzeń, reaktor automatycznie przekazuje sterowanie do wbudowanego algorytmu Lokalnego Automatycznego Regulatora (automatic_regulator). Po przetworzeniu logiki sterowania i przeliczeniu termodynamiki, stan obiektu jest synchronicznie wypychany do bazy InfluxDB pomiarem rbmk_reactor_metrics przy użyciu API w trybie SYNCHRONOUS.

4. Matematyczny model fizyki rdzenia i termohydrauliki

Serce symulatora stanowi klasa Reactor (plik schemas.py). Co sekundę wykonywany jest pełny cykl obliczeniowy aktualizujący stan reaktora.

4.1. Bilans cieplny i generowanie pary

Układ termodynamiczny wyznacza temperaturę chłodziwa na wylocie z rdzenia (outlet_temp_c) oraz ułamek objętościowy pary wodnej (v_steam). Przyrost temperatury chłodziwa determinowany jest przez stosunek generowanej mocy cieplnej do natężenia przepływu wody chłodzącej, skalowany stałą pojemności cieplnej układu ($k_{termo} = 197.0$)

$$\Delta T = k_{termo} \cdot \frac{P_{thermal}}{Q_{coolant}}$$

$$T_{outlet} = T_{inlet} + \Delta T$$

Nominalna temperatura wrzenia wody w kanałach technologicznych wynosi 284.0 °C. Po osiągnięciu tej temperatury rozpoczyna się generacja fazy parowej (ułamek pustynnego v_steam), modelowana równaniem:

$$V_{steam} = \left(3.068 \cdot \frac{P_{thermal}}{Q_{coolant}} \right) - (M_{subcooling} \cdot 0.005)$$

gdzie $M_{subcooling}$ reprezentuje margines niedogrzaną chłodziwa na wlocie do rdzenia.

4.2. Dynamiczne sprzężenia zwrotne i przyrost reaktywności

Całkowity przyrost reaktywności ($\Delta\rho$, zmienna reactivity_delta) jest sumą algebraiczną wpływu operatora oraz fizycznych cech rdzenia:

$$\Delta\rho = \rho_{base} + \rho_{void} + \rho_{xenon} + \rho_{rods}$$

- a) Dodatni efekt pustynny (Void Effect): Kluczowa niestabilność fizyczna modelu. Wzrost ilości pary (zmniejszenie gęstości moderatora wodnego) powoduje gwałtowny wzrost reaktywności:

$$\rho_{void} = V_{steam} \cdot 4.5$$

- b) Zatrucie ksenonowe (Xenon Poisoning): Dynamiczny proces akumulacji nuklidu Ksenonu-135 (`xenon_level`), pełniącego rolę trucizny neutronowej o silnym przekroju czynnym na wychwyt:

$$\rho_{\text{xenon}} = -4.0 \cdot X_{\text{level}}$$

- c) Wpływ układu regulacji (Control Rods): Pozycja prętów definiuje pochłanianie neutronów. Pręty całkowicie wsunięte (`orm_value`) generują potężną ujemną reaktywność, wygaszając reaktor, podczas gdy pręty automatyczne (`partially_inserted`) stabilizują stan krytyczny:

$$\rho_{\text{rods}} = -(ORM \cdot 0.06) + (P_{\text{inserted}} \cdot 0.02)$$

Zmiana mocy cieplnej reaktora w czasie oparta jest na wyliczonej reaktywności oraz stałej czasowej reaktora τ (tau):

$$P_{\text{thermal}(t+\Delta t)} = P_{\text{thermal}(t)} + \left(\frac{\Delta \rho \cdot P_{\text{thermal}(t)}}{\tau} \right) \cdot \Delta t$$

5. Projekt interfejsu HMI Grafana i logika operatorska

Pulpit sterowniczy w Grafanie został zoptymalizowany pod kątem natychmiastowej oceny bezpieczeństwa reaktora.

5.1. Dychotomiczny podział prezentacji prętów kontrolnych

W celu uniknięcia błędów interpretacyjnych przez operatora, zrezygnowano z jednego wskaźnika procentowego. Zaimplementowano dwa osobne, sprzężone panele typu Bar Gauge wyskalowane w jednostkach bezwzględnych – sztukach (skala od 0 do 211 prętów):

- a) Partially Inserted Rods (Pcs): Reprezentuje liczbę prętów pracujących w reżimie dynamicznej regulacji i automatyki (`partially_inserted`). Wizualizowany w kolorze zielonym (bezpiecznym). Zapytanie Flux:

```
from(bucket: "reactor_test")
  |> range(start: v.timeRangeStart, stop: v.timeRangeStop)
  |> filter(fn: (r) => r._measurement == "reactor_metrics_test2")
  |> filter(fn: (r) => r._field == "partially_inserted")
  |> last()
  |> map(fn: (r) => ({ r with _value: math.round(x: (float(v: r._value) / 100.0) * 211.0) })))
```

- b) Fully Inserted / SCRAM (Pcs): Pokazuje liczbę prętów, które osiągnęły dolne położenie krańcowe (pełne zanurzenie ochronne). Wykorzystuje matematyczne różnicowanie zmiennych bez zmian backendowych:

```
from(bucket: "reactor_test")
  > range(start: v.timeRangeStart, stop: v.timeRangeStop)
  > filter(fn: (r) => r._measurement == "reactor_metrics_test2")
  > filter(fn: (r) => r._field == "orm_value" or r._field == "partially_inserted")
  > last()
  > pivot(rowKey:["_time"], columnKey: ["_field"], valueColumn: "_value")
  > map(fn: (r) => {
    partial_pcs = math.round(x: (float(v: r.partially_inserted) / 100.0) * 211.0)
    return { _time: r._time, _value: float(v: r.orm_value) - partial_pcs }
  })
```

6. Zaawansowana detekcja anomalii za pomocą sieci neuronowej Deep Learning (dataset.py, model.py, train.py)

W celu implementacji warstwy predykcyjnej, w module model zaimplementowano głęboką architekturę rekurencyjną Autoencodera opartą na komórkach LSTM/GRU.

6.1. Mechanizm okna przesuwne i przetwarzania danych (dataset.py)

Dane historyczne reaktora zapisane w formacie binarnym Apache Parquet (Influx_RBML_data.parquet) są ładowane i oczyszczane z wartości anomalnych i brakujących (NaN). Ponieważ sieci LSTM wymagają stałych sekwencji czasowych, klasa TimeSeriesDataset implementuje algorytm okna przesuwne (Sliding Windows):

- Każda próbka ucząca stanowi macierz o wymiarach [sequence_length, num_features] (w konfiguracji domyślnej: [50, 12] – co odpowiada 50 sekundom ciągłego monitoringu 12 sensorów).
- Nakładające się okna przesuwne maksymalizują stopień wykorzystania danych z szeregów czasowych.
- Dane poddawane są normalizacji statystycznej co centruje średnią wokół 0 i sprowadza odchylenie standardowe do 1, optymalizując proces spadku gradientu w sieci LSTM.

6.2. Architektura sieci neuronowej LSTM Autoencoder (model.py)

Model realizuje paradygmat uczenia nienadzorowanego (Unsupervised Learning). Zadaniem sieci jest rekonstrukcja sygnału wejściowego na wyjściu. Sieć składa się z dwóch głównych komponentów: Encodera i Decodera.

1. Encoder (Koder): Wielowarstwowa sieć LSTM (domyślnie num_layers=2 z ukrytą warstwą hidden_size=64 oraz regularyzacją dropout=0.2). Encoder analizuje sekwencję 50 kroków czasowych dla wszystkich sensorów i kompresuje je do ukrytego wektora stanu (tzw. przestrzeni ukrytej / Bottleneck), który reprezentuje esencję zachowania reaktora.
2. Decoder (Dekoder): Przyjmuje skompresowany wektor stanu i próbuje odtworzyć (zrekonstruować) pierwotną, pełną sekwencję czasową o wymiarach [50, 12].
3. Warstwa liniowa (Linear Projection): Mapuje wyjście z dekodera bezpośrednio na oryginalną przestrzeń cech fizycznych.

6.3. Proces uczenia i funkcja straty (train.py)

Sieć trenowana jest wyłącznie na danych reprezentujących normalną, stabilną i bezpieczną pracę reaktora. Dzięki temu model uczy się perfekcyjnie rekonstruować stany standardowe. Jako funkcję straty wykorzystano błąd średniokwadratowy MSE (Mean Squared Error):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

Funkcja MSE penalizuje duże błędy kwadratowo, co jest pożądane w detekcji anomalii. W pętli treningowej zaimplementowano zaawansowane mechanizmy:

- Gradient Clipping: Obcinanie normy gradientu do poziomu 1.0 w celu eliminacji problemu eksplodującego gradientu (exploding gradients) typowego dla głębokich sieci LSTM.
- Early Stopping: Monitorowanie straty na zbiorze walidacyjnym (Validation Loss) z cierpliwością ustawioną na 10 epok, co zapobiega zjawisku przeuczenia sieci (overfitting).
- Dynamiczna redukcja współczynnika uczenia: Użycie optimizera Adam wraz z harmonogramem ReduceLROnPlateau, który zmniejsza współczynnik uczenia (learning_rate) o połowę w momencie wejścia straty w fazę plateau.

6.4. Matematyczna klasyfikacja anomalii i wyznaczanie progu (model_to_db.py)

Detekcja anomalii opiera się na błędzie rekonstrukcji. Podczas inferencji (model_to_db.py), nowe dane z reaktora są przepuszczane przez model.

- Jeśli reaktor pracuje normalnie -> sieć bez problemu odtwarza sygnały -> błąd MSE jest bliski zeru.
- Jeśli w reaktorze dochodzi do nieliniowej awarii (np. uderzenie pary, awaria pomp, zatrucie) -> stany te różnią się od danych uczących -> sieć nie potrafi ich poprawnie odtworzyć -> błąd MSE gwałtownie rośnie.

Próg alarmowy (Threshold) wyznaczany jest statystycznie przy użyciu 95. percentyla błędów rekonstrukcji na zbiorze walidacyjnym:

$$\text{Threshold} = P_{95}(\text{MSE}_{\text{val}})$$

Oznacza to, że top 5% próbek o najwyższym uchybie w stanie nominalnym definiuje granicę czułości. Każda sekwencja testowa, dla której $\text{MSE} > \text{Threshold}$ zostaje automatycznie oznaczona w bazie InfluxDB jako stan anomalny ($\text{is_anomaly} = 1$).

7. Środowisko konteneryzacji i orkiestracji Docker (docker-compose.yml)

W celu zapewnienia pełnej niezależności od systemów operacyjnych oraz wsparcia dla akceleracji sprzętowej, cały ekosystem projektu został zaimplementowany w architekturze wielokontenerowej Docker (docker-compose.yml).

Konfiguracja definiuje dwie główne usługi spięte mostkową siecią wirtualną (model-network):

1. **pytorch-model-dev**: Kontener aplikacyjny oparty na obrazie środowiska uczenia głębokiego PyTorch. Kontener posiada zaawansowane wsparcie dla orkiestracji zasobów sprzętowych GPU za pomocą sterownika NVIDIA CUDA (driver: nvidia), co umożliwia przeniesienie obliczeń tensorowych bezpośrednio do pamięci VRAM karty graficznej, generując 10-100x przyspieszenie procesu uczenia. Wykorzystuje mechanizm Docker Volumes do mapowania kodu źródłowego i trwałych plików Parquet na dysku hosta.
2. **influxdb-dev**: Izolowany kontener oficjalnego obrazu bazy danych InfluxDB (wersja 2.7-alpine), wystawiający port komunikacyjny 8086 oraz zabezpieczony szyfrowanymi zmiennymi środowiskowymi dla administratora układu. Dane bazy są trwale mapowane na niezależny wolumen wolnościowy influxdb-data.

8. Scenariusz demonstracyjny (Przebieg testu)

System został zwalidowany podczas testu dynamicznego składającego się z trzech faz:

1. **Faza 1: Krytyczność nominalna**: Reaktor generuje stabilne 3200 MW. Reaktywność wynosi 0.0. Sieć neuronowa generuje minimalny błąd rekonstrukcji ($MSE = 0.005$), system HMI wskazuje stan stabilny.
2. **Faza 2: Kryzys termohydrauliczny i detekcja anomalii**: Symulowane jest drastyczne odcięcie pompy chłodzącej (coolant_flow_m3h spada poniżej 30000). Temperatura przekracza $284^{\circ}C$, generując ułamek pary v_steam. Dodatni efekt pustynny destabilizuje reaktor – moc niekontrolowanie rośnie. Ponieważ ten stan drastycznie odbiega od stanów nominalnych, błąd rekonstrukcji sieci neuronowej przekracza wyznaczony próg statystyczny ($MSE > Threshold$). Na ekranie Grafany wskaźnik anomalii natychmiast zmienia stan na alarmowy.
3. **Faza 3: Inicjacja procedury AZ-5**: Operator, ostrzeżony przez system HMI oraz model głębokiego uczenia, uruchamia procedurę wygaszania. Prawy wskaźnik prętów zeruje się, a lewy panel (Fully Inserted) błyskawicznie wypełnia się do wartości 211 sztuk w kolorze czerwonym. Reaktywność spada głęboko poniżej zera, a moc bezpiecznie wygasza się do poziomu szczątkowego ciepła powyłączeniowego (160 MW).

9. Podsumowanie i wnioski

Zapożyczony i zaimplementowany symulator z powodzeniem integruje zaawansowane zagadnienia inżynierii oprogramowania, fizyki jądrowej oraz sztucznej inteligencji.

Użycie bazy InfluxDB w połączeniu z brokerem Redis i serwerem FastAPI zapewniło stabilność przesyłu i przetwarzania metryk z częstotliwością 1 HZ bez jakichkolwiek opóźnień sieciowych. Zastosowanie transformacji języka Flux przeniosło ciężar wizualizacji bezpośrednio na bazę danych. Największą innowacją systemu jest zaimplementowany model Autoencodera LSTM w środowisku PyTorch, który wykazał się niezwykle czułością na nieliniowe fluktuacje parametrów termohydraulicznych reaktora, poprawnie klasyfikując awarie i przesyłając wyniki predykcji z powrotem do bazy danych.

Całość tworzy nowoczesny, kompletny system HMI klasy przemysłowej.

10. Oświadczenie o wykorzystaniu technologii asystujących i narzędzi AI

W procesie redakcji, korekty językowej oraz ustrukturyzowania niniejszej dokumentacji technicznej wykorzystano zaawansowany model językowy Gemini (opracowany przez firmę Google). Dodatkowo, w celu optymalizacji i weryfikacji warstwy analitycznej oraz modeli uczenia maszynowego, wykorzystano pakiety Cloud Somnet 4.6 oraz HQ 4.5. Narzędzia te pełniły funkcję asystującą, wspierając zespół w integracji opisów architektury oraz zapewnieniu spójności stylistycznej i merytorycznej dokumentu. Wszystkie koncepcje inżynierskie, logika algorytmów oraz implementacja kodu źródłowego systemu są wynikiem samodzielnej pracy autorów.